

FastAPI Best Practices

Interview Q&A

32 In-Depth Questions on Backend Best Practices, SOLID Principles for API Design, API Paradigms (REST/GraphQL/gRPC/Real-Time), Production Deployment, System Design Fundamentals, and Real-Time Failure Scenarios

Each question includes a real-world problem developers run into, the reasoning behind the fix, working code examples, and a concise interview-ready answer.

Covers: Project Architecture & SOLID | Async & Database Pitfalls | Security | Docker & Deployment | System Design Fundamentals | REST vs GraphQL vs gRPC | Production Incident Scenarios

Contents

Project Structure & Design Principles (1-5)

- Q1. How should you structure a large FastAPI project, and why does a flat structure fail in production?
- Q2. How does the Dependency Inversion Principle apply to FastAPI's Depends() system, and what problem does it solve in real projects?
- Q3. What is the correct way to manage configuration and secrets with Pydantic Settings, and what mistakes do developers commonly make?
- Q4. Why is it dangerous to use def instead of async def for I/O-bound routes, and what actually happens under the hood when you get it wrong?
- Q5. How do you design a custom exception hierarchy so error handling doesn't turn into scattered try/except blocks across every route?

Backend Development Practices (6-10)

- Q6. What is the N+1 query problem, and how does it silently appear in FastAPI + SQLAlchemy applications?
- Q7. How should database sessions be managed to avoid connection leaks and pool exhaustion under load?
- Q8. How do you design idempotent APIs, and why does this matter for payment or order-creation endpoints specifically?
- Q9. What security mistakes are commonly found in FastAPI applications during code review, and how do you fix each?
- Q10. How do you apply the Open/Closed and Interface Segregation principles when building a service that supports multiple payment providers (Stripe, Razorpay, PayPal)?

Deployment & Production Practices (11-13)

- Q11. How do you write a production-grade Dockerfile for FastAPI, and what mistakes make images slow or insecure?
- Q12. What do health checks and readiness probes actually need to verify, and why does a naive /health endpoint cause outages during deployment?
- Q13. How do you scale a FastAPI application to handle roughly 1 million requests per day, and where do teams usually get the bottleneck wrong?

Real-Time Case Scenarios (14-15)

- Q14. Scenario: Your FastAPI service's memory usage grows steadily over several hours until it's OOM-killed and restarted by Kubernetes. How do you diagnose and fix this?
- Q15. Scenario: During a flash sale, your /checkout endpoint starts returning 500 errors and the database shows 'too many connections'. Walk through the root cause and the fix.

Designing Good APIs -- SOLID Principles Explained (16-20)

- Q16. (S) How does the Single Responsibility Principle guide good REST endpoint design, and what does a violation look like in practice?
- Q17. (O) How does the Open/Closed Principle apply to evolving an API without breaking existing clients?
- Q18. (L) What does the Liskov Substitution Principle mean for Pydantic schemas and interchangeable service implementations?
- Q19. (I) How does Interface Segregation shape request/response schema design and service interfaces so clients aren't forced to depend on things they don't need?

Q20. (D) Beyond Depends(), how does Dependency Inversion shape the overall architecture of an API from routers down to external systems?

System Design Fundamentals Every Backend Engineer Should Know (21-27)

Q21. What is the difference between horizontal and vertical scaling, and why does statelessness matter for scaling an API?

Q22. What are the main caching strategies (cache-aside, write-through, write-behind), and when should each be used?

Q23. What is the CAP theorem, and how does it practically influence database choices in a backend system?

Q24. What is the difference between database read replicas and sharding, and when would you reach for each?

Q25. What are message queues used for in a backend system, and what's the difference between point-to-point and pub-sub messaging?

Q26. What are the main rate-limiting algorithms, and how do token bucket and sliding window differ in practice?

Q27. What is an API Gateway, and which responsibilities belong there versus inside individual backend services?

API Paradigms: REST vs GraphQL vs gRPC vs Real-Time (28-32)

Q28. What are the fundamental differences between REST, GraphQL, and gRPC, and how do you decide which one to use for a given service?

Q29. What are over-fetching and under-fetching, and how does GraphQL solve them compared to REST?

Q30. How do you choose between WebSockets, Server-Sent Events (SSE), and polling for real-time features in a backend?

Q31. Scenario: You're designing a system with a public-facing API, internal microservices, and a live dashboard. How do you decide which API style to use where?

Project Structure & Design Principles (1-5)

Q1. How should you structure a large FastAPI project, and why does a flat structure fail in production?

A common beginner mistake is putting everything -- routes, DB logic, business logic, and schemas -- inside a single main.py file. It works for a demo but becomes unmanageable once the project crosses 20-30 endpoints: merge conflicts increase, testing becomes hard, and unrelated features get coupled together.

A production-grade layout separates concerns into distinct layers, similar to the classic Controller - Service - Repository pattern used in Spring Boot or NestJS:

```
app/
├── api/
│   └── v1/
│       ├── routers/
│       │   ├── users.py           # Route definitions only
│       │   ├── orders.py
│       │   └── deps.py           # Shared dependencies
│       ├── services/
│       │   ├── user_service.py    # Business logic
│       │   └── order_service.py
│       ├── repositories/
│       │   ├── user_repository.py # DB access only
│       │   └── order_repository.py
│       ├── models/               # SQLAlchemy ORM models
│       ├── schemas/             # Pydantic request/response models
│       └── core/
│           ├── config.py         # Settings
│           ├── security.py
│           └── exceptions.py
│       ├── db/
│       │   └── session.py
│       └── main.py               # App factory, includes routers
```

The router only receives HTTP input and returns HTTP output. The service layer holds business rules (e.g. "a discount cannot exceed 50%"). The repository layer is the only place that talks to the database. This mirrors the Single Responsibility Principle (SRP) from SOLID -- each module has exactly one reason to change.

A concrete daily-development problem this solves: if you later swap PostgreSQL for MongoDB, or add caching, you only touch the repository layer -- routers and services remain untouched.

Interview Answer:

A scalable FastAPI project separates routers, services, and repositories into distinct layers so that HTTP handling, business logic, and data access evolve independently. This directly applies the Single Responsibility and Dependency Inversion principles from SOLID, and makes the codebase testable, since services can be unit-tested with mocked repositories instead of a real database.

Q2. How does the Dependency Inversion Principle apply to FastAPI's Depends() system, and what problem does it solve in real projects?

A very common anti-pattern seen in real codebases is instantiating a database session or an external client (e.g. a payment gateway SDK) directly inside a route:

```
@app.get("/orders/{id}")
def get_order(id: int):
    db = SessionLocal()          # tightly coupled, hard to mock
    order = db.query(Order).get(id)
    db.close()
    return order
```

This violates the Dependency Inversion Principle: the high-level route handler depends directly on a low-level, concrete implementation (`SessionLocal`). Testing this route requires a real database connection -- unit testing becomes integration testing.

The fix is to depend on an abstraction, injected via `Depends()`, so the concrete implementation can be swapped at runtime or in tests:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/orders/{id}")
def get_order(id: int, db: Session = Depends(get_db)):
    return db.query(Order).get(id)

# In tests:
app.dependency_overrides[get_db] = lambda: fake_session
```

This one change makes the route unit-testable in milliseconds using an in-memory fake, instead of spinning up Postgres for every test run in CI.

Interview Answer:

`Depends()` lets FastAPI apply Dependency Inversion: route handlers depend on an abstraction (a callable that yields a resource) rather than a concrete class. This makes `dependency_overrides` possible in tests, keeps business code decoupled from infrastructure, and is the single most important habit for testable FastAPI code.

Q3. What is the correct way to manage configuration and secrets with Pydantic Settings, and what mistakes do developers commonly make?

A very common production incident: someone hardcodes `SECRET_KEY = "mysecret123"` directly in code, commits it to Git, and it ends up on GitHub. Another frequent mistake is reading `os.environ.get()` scattered across 15 different files with no validation, so a missing variable only fails at runtime, in production, at 2 AM.

The correct approach centralizes configuration in one place using `pydantic-settings`, so missing or malformed environment variables fail fast at application startup, not mid-request:

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    redis_url: str = "redis://localhost:6379"
    environment: str = "development"
```

```

class Config:
    env_file = ".env"

@lru_cache
def get_settings() -> Settings:
    return Settings()

# usage
@app.get("/health")
def health(settings: Settings = Depends(get_settings)):
    return {"env": settings.environment}

```

@lru_cache ensures Settings() is constructed only once per process, avoiding repeated disk/env reads. Injecting settings via Depends() (instead of importing a global) also means tests can override configuration cleanly.

Interview Answer:

Centralize configuration in a single Pydantic Settings class loaded from environment variables, never hardcode secrets, and inject settings through Depends() rather than importing a module-level global. This gives fail-fast validation at startup and keeps secret management out of version control.

Q4. Why is it dangerous to use def instead of async def for I/O-bound routes, and what actually happens under the hood when you get it wrong?

This is one of the most common real-world FastAPI production bugs. Developers assume marking every route async def automatically makes it fast -- but the opposite mistake causes silent, severe performance degradation: calling a blocking library inside an async def route.

```

# DANGEROUS -- blocks the entire event loop
@app.get("/report")
async def generate_report():
    data = requests.get("https://slow-api.com/data") # blocking call!
    time.sleep(2) # blocking!
    return data.json()

```

FastAPI runs async def routes directly on the single event loop. requests.get() and time.sleep() are synchronous, blocking calls -- while one is waiting, the entire event loop is frozen, so every other concurrent request on that worker stalls too. Under load, this turns a 50ms endpoint into one that appears to hang for seconds.

There are two correct fixes depending on the operation type:

Situation	Correct Fix
Library has an async version (httpx, asyncpg)	Use async def with await httpx_client.get(...)
Library is sync-only (e.g. legacy SDK) or CPU-bound work	Use plain def -- FastAPI automatically runs it in a threadpool

```

# Correct: use an async-native client
@app.get("/report")
async def generate_report():
    async with httpx.AsyncClient() as client:
        data = await client.get("https://slow-api.com/data")
    return data.json()

# Correct: sync-only code -- let FastAPI offload to threadpool

```

```
@app.get("/legacy-report")
def generate_legacy_report():
    return legacy_sdk.fetch_report() # runs in threadpool, doesn't block loop
```

Interview Answer:

async def only helps if every awaited call inside it is truly non-blocking. Mixing blocking calls (requests, time.sleep, sync DB drivers) into async def freezes the entire event loop for all concurrent requests. If a dependency has no async version, define the route with plain def so FastAPI runs it in a threadpool instead.

Q5. How do you design a custom exception hierarchy so error handling doesn't turn into scattered try/except blocks across every route?

A frequent code smell in growing FastAPI codebases: every route repeats near-identical try/except blocks with slightly different HTTPException messages, making error handling inconsistent across the API and hard to change globally.

```
# Repeated everywhere -- hard to maintain
@app.get("/users/{id}")
def get_user(id: int):
    user = db.query(User).get(id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user
```

A cleaner design defines domain-specific exceptions in the service layer, decoupled from HTTP entirely, and registers a single global handler that translates them to HTTP responses. This follows the Open/Closed Principle -- you can add new domain exceptions without modifying existing route code.

```
# exceptions.py
class DomainException(Exception):
    pass

class UserNotFoundException(DomainException):
    def __init__(self, user_id: int):
        self.user_id = user_id

# service layer raises domain exceptions, knows nothing about HTTP
def get_user_or_raise(db, user_id: int):
    user = db.query(User).get(user_id)
    if not user:
        raise UserNotFoundException(user_id)
    return user

# main.py -- one global translation point
@app.exception_handler(UserNotFoundException)
async def user_not_found_handler(request, exc: UserNotFoundException):
    return JSONResponse(
        status_code=404,
        content={"error": "USER_NOT_FOUND", "user_id": exc.user_id},
    )
```

Interview Answer:

Define a hierarchy of domain-specific exceptions that carry no HTTP knowledge, raise them from the service layer, and translate them to HTTP responses in one place using @app.exception_handler. This keeps error semantics consistent across the whole API and lets you change response formats without touching every route.

Backend Development Practices (6-10)

Q6. What is the N+1 query problem, and how does it silently appear in FastAPI + SQLAlchemy applications?

This is one of the most common performance bugs found during code review. It looks fine in development with 5 rows of test data, then causes timeouts in production with 10,000 rows.

```
# Looks innocent, but is an N+1 query bug
@app.get("/orders")
def list_orders(db: Session = Depends(get_db)):
    orders = db.query(Order).all()          # 1 query
    return [
        {"id": o.id, "customer": o.customer.name} # +1 query PER order!
        for o in orders
    ]
```

Accessing `o.customer` triggers a lazy-loaded query for every single order in the loop. 1,000 orders means 1,001 database round trips instead of 2. The fix is eager loading -- telling SQLAlchemy to fetch related rows in the same query (or one extra query) up front:

```
from sqlalchemy.orm import joinedload

@app.get("/orders")
def list_orders(db: Session = Depends(get_db)):
    orders = (
        db.query(Order)
        .options(joinedload(Order.customer)) # single JOIN query
        .all()
    )
    return [{"id": o.id, "customer": o.customer.name} for o in orders]
```

In async SQLAlchemy, the equivalent is `selectinload()` for one-to-many relationships, which avoids row duplication that `joinedload()` can cause on large collections.

Interview Answer:

The N+1 problem happens when accessing a lazy-loaded relationship inside a loop triggers one extra query per row. It's invisible in local testing with tiny datasets and catastrophic in production. Always use `joinedload()` or `selectinload()` when you know a related object will be accessed for every row in a result set.

Q7. How should database sessions be managed to avoid connection leaks and pool exhaustion under load?

A real production incident pattern: an API works fine at low traffic, then starts throwing `QueuePool limit of size 5 overflow 10 reached` errors as traffic grows. The root cause is almost always a session that is opened but never properly closed on every code path, especially when an exception is raised mid-request.

```
# BUG: if db.query() raises, db.close() is never called
def get_db():
    db = SessionLocal()
    return db # session leaks on every unhandled exception
```


The generator-based dependency pattern with try/finally is the correct approach, because FastAPI guarantees the finally block runs even if the route raises an exception:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()      # always runs, even on exception
```

For production workloads, also tune the pool explicitly rather than relying on defaults, and set `pool_pre_ping` to avoid using dead connections after a DB restart or network blip:

```
engine = create_engine(
    DATABASE_URL,
    pool_size=20,
    max_overflow=10,
    pool_pre_ping=True,      # validates connection before use
    pool_recycle=1800,      # recycle connections every 30 min
)
```

Interview Answer:

Always manage sessions with a generator dependency using try/finally so the connection is released even when an exception occurs mid-request. Under real traffic, also tune `pool_size`, `max_overflow`, and enable `pool_pre_ping` -- default pool settings are tuned for development, not production concurrency.

Q8. How do you design idempotent APIs, and why does this matter for payment or order-creation endpoints specifically?

A classic real-world bug: a user on a slow mobile network double-taps "Pay Now". The client retries the request after a timeout, but the first request actually succeeded on the server. Without idempotency, this creates two charges for one purchase -- a serious production incident, not a theoretical edge case.

The fix is to require an Idempotency-Key header from the client, and store the result of the first request keyed by that value. Any retry with the same key returns the cached result instead of re-executing the operation:

```
@app.post("/payments")
async def create_payment(
    payload: PaymentRequest,
    idempotency_key: str = Header(...),
    db: Session = Depends(get_db),
    redis: Redis = Depends(get_redis),
):
    cached = await redis.get(f"idem:{idempotency_key}")
    if cached:
        return json.loads(cached)      # safe retry -- no double charge

    result = await payment_service.charge(db, payload)
    await redis.set(f"idem:{idempotency_key}", json.dumps(result), ex=86400)
    return result
```

This same pattern applies to any operation with real-world side effects: sending emails, creating orders, triggering webhooks. As a design rule, POST endpoints that create billable or irreversible side effects should always support idempotency keys in production.

Interview Answer:

Idempotency prevents duplicate side effects (like double charges) when clients retry requests after timeouts, which happens constantly on real networks. Implement it by having the client send a unique Idempotency-Key, storing the first response against that key in Redis, and short-circuiting any retry with the same key.

Q9. What security mistakes are commonly found in FastAPI applications during code review, and how do you fix each?

Security issues rarely come from exotic attacks -- they come from small, repeatable mistakes. The most frequent ones found in real audits:

Mistake	Fix
Storing plaintext passwords	Hash with bcrypt/argon2 via passlib before storing
Returning ORM objects directly (leaks internal fields)	Always return a Pydantic response_model, never the DB model
Wildcard CORS in production: allow_origins=["*"]	Whitelist explicit frontend origins only
No request size / rate limits on public endpoints	Add SlowAPI rate limiting and request body size limits
Long-lived JWTs with no revocation	Short-lived access tokens + refresh tokens + a Redis blacklist

```
# Mistake: leaks every DB column, including password_hash
@app.get("/users/{id}")
def get_user(id: int, db: Session = Depends(get_db)):
    return db.query(User).get(id)    # returns raw ORM object

# Fix: response_model filters output to only declared fields
class UserOut(BaseModel):
    id: int
    name: str
    email: str
    # password_hash intentionally omitted

@app.get("/users/{id}", response_model=UserOut)
def get_user(id: int, db: Session = Depends(get_db)):
    return db.query(User).get(id)    # extra fields silently stripped
```

The response_model fix is especially important because it's easy to forget: adding a new sensitive column to the ORM model later (e.g. two_factor_secret) would otherwise leak automatically through every endpoint that returns that model.

Interview Answer:

Hash passwords with bcrypt/argon2, never expose raw ORM objects (always use response_model to whitelist output fields), restrict CORS to known origins, rate-limit public endpoints, and keep JWTs short-lived with a revocation mechanism. Most FastAPI security incidents come from these repeatable, well-known mistakes rather than novel exploits.

Q10. How do you apply the Open/Closed and Interface Segregation principles when building a service that supports multiple payment providers (Stripe, Razorpay,

PayPal)?

A common maintainability problem: as a company adds more payment providers, developers add if/elif blocks checking `provider == "stripe"` throughout the codebase. Every new provider means editing existing, already-tested code -- a direct violation of the Open/Closed Principle (code should be open for extension, closed for modification).

```
# Violates Open/Closed -- must edit this function for every new provider
def charge(provider: str, amount: float):
    if provider == "stripe":
        return stripe.Charge.create(amount=amount)
    elif provider == "razorpay":
        return razorpay_client.payment.create(amount=amount)
    # adding paypal means modifying this function again
```

The fix defines a small abstract interface that every provider implements, and the calling code depends only on that interface (Interface Segregation -- the interface exposes only what callers need, nothing provider-specific):

```
from abc import ABC, abstractmethod

class PaymentProvider(ABC):
    @abstractmethod
    async def charge(self, amount: float, currency: str) -> dict: ...

class StripeProvider(PaymentProvider):
    async def charge(self, amount: float, currency: str) -> dict:
        return await stripe_client.create_charge(amount, currency)

class RazorpayProvider(PaymentProvider):
    async def charge(self, amount: float, currency: str) -> dict:
        return await razorpay_client.create_payment(amount, currency)

# FastAPI dependency picks the right implementation
def get_payment_provider(provider: str = "stripe") -> PaymentProvider:
    return {"stripe": StripeProvider(), "razorpay": RazorpayProvider()}[provider]

@app.post("/pay")
async def pay(amount: float, provider: PaymentProvider = Depends(get_payment_provider)):
    return await provider.charge(amount, "USD")
```

Adding PayPal now means writing one new class -- zero changes to the route, the dependency signature, or existing provider classes.

Interview Answer

Define a small abstract base class (`PaymentProvider`) representing only the operations callers actually need, and have each provider implement it independently. New providers are added by writing new classes, not editing existing ones -- satisfying both the Open/Closed Principle and Interface Segregation, and making the route code provider-agnostic.

Deployment & Production Practices (11-13)

Q11. How do you write a production-grade Dockerfile for FastAPI, and what mistakes make images slow or insecure?

Common mistakes seen in real Dockerfiles: using the full python:3.12 image (900MB+) instead of a slim variant, running the app as root, reinstalling all dependencies on every code change because COPY . . happens before pip install, and baking secrets into image layers.

```
# Multi-stage build -- smaller, faster, safer
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .

# Run as non-root user
RUN useradd -m appuser && chown -R appuser /app
USER appuser

ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

Two important details: requirements.txt is copied and installed before the rest of the source code, so Docker's layer cache is reused and dependencies aren't reinstalled on every source change; and the container runs as appuser instead of root, limiting blast radius if the container is ever compromised.

In production behind Kubernetes or ECS, Uvicorn is typically run with Gunicorn as a process manager for multiple worker processes and automatic restarts on crash:

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

Interview Answer:

Use multi-stage builds on a slim base image, order Dockerfile instructions so dependency installation is cached separately from source code changes, and never run the container as root. In production, Gunicorn typically manages multiple Uvicorn worker processes for resilience and CPU utilization across cores.

Q12. What do health checks and readiness probes actually need to verify, and why does a naive /health endpoint cause outages during deployment?

A common deployment bug: a Kubernetes liveness probe hits /health, which just returns {"status": "ok"} unconditionally. The pod passes health checks and receives live traffic even though its database connection pool is exhausted or Redis is unreachable -- users see 500 errors while Kubernetes thinks everything is fine.

Liveness and readiness serve different purposes and should not be the same check:

Probe	Question it answers	Failure action
Liveness	Is the process alive / not deadlocked?	Kubernetes restarts the pod
Readiness	Can this pod actually serve traffic right now?	Kubernetes stops routing traffic to it, but does not restart it

```

@app.get("/health/live")
async def liveness():
    return {"status": "alive"}    # cheap, no dependencies -- just proves process responds

@app.get("/health/ready")
async def readiness(db: Session = Depends(get_db), redis: Redis = Depends(get_redis)):
    try:
        db.execute(text("SELECT 1"))
        await redis.ping()
    except Exception:
        raise HTTPException(status_code=503, detail="Dependencies unavailable")
    return {"status": "ready"}

```

This distinction matters most during rolling deployments and dependency outages: if the database briefly restarts, readiness should fail (so the load balancer stops sending traffic) but liveness should still pass (so Kubernetes doesn't pointlessly restart a perfectly healthy application process).

Interview Answer:

Liveness checks whether the process itself is alive and should have no external dependencies. Readiness checks whether the pod can currently serve traffic, and should actively verify database and cache connectivity. Conflating the two causes either unnecessary pod restarts or, worse, traffic being routed to pods that can't actually serve requests.

Q13. How do you scale a FastAPI application to handle roughly 1 million requests per day, and where do teams usually get the bottleneck wrong?

1M requests/day averages to about 12 requests/second, which a single well-tuned instance can often handle -- but real traffic isn't evenly distributed. Peak hours might see 10x that average, so the design target is really peak concurrency, not the daily average. Teams often over-invest in horizontal scaling while ignoring cheaper, more impactful fixes first.

A practical order of optimization, from cheapest/highest-impact to most expensive:

Step	Action	Typical impact
1	Add DB indexes on frequently filtered/joined columns	Often 10-100x on slow queries
2	Cache hot, rarely-changing reads in Redis with a TTL	Removes repeated DB load entirely
3	Fix N+1 queries and use async DB drivers (asyncpg)	Reduces per-request latency and thread usage
4	Run multiple Uvicorn workers per instance (CPU cores)	Uses all available CPU on one machine
5	Horizontally scale instances behind a load balancer	Handles traffic beyond single-machine capacity
6	Add a CDN / edge cache for static or cacheable responses	Offloads traffic before it reaches the app

```

# Example: caching a hot, expensive read
@app.get("/products/popular")
async def popular_products(redis: Redis = Depends(get_redis), db=Depends(get_db)):
    cached = await redis.get("popular_products")

```

```

if cached:
    return json.loads(cached)

result = await db.execute(select(Product).order_by(Product.sales.desc()).limit(20))
products = [p.dict() for p in result.scalars().all()]
await redis.set("popular_products", json.dumps(products), ex=300) # 5 min TTL
return products

```

In practice, most FastAPI performance incidents are traced back to step 1 or 2 (missing indexes, no caching) rather than needing more servers -- scaling horizontally before fixing an N+1 query just multiplies the same inefficient database load across more machines.

Interview Answer:

Scale in order of cost: fix indexes and N+1 queries first, then add caching for hot reads, then use async drivers, then add more Uvicorn workers per instance, and only then scale horizontally with a load balancer. Most real bottlenecks are inefficient queries, not insufficient hardware -- adding servers without fixing the query pattern just scales the waste.

Real-Time Case Scenarios (14-15)

Q14. Scenario: Your FastAPI service's memory usage grows steadily over several hours until it's OOM-killed and restarted by Kubernetes. How do you diagnose and fix this?

This is a classic slow-leak incident, distinct from a crash -- the service works fine for hours, then dies, then works fine again after restart, making it easy to dismiss as "random" if you don't investigate systematically.

A methodical diagnosis approach:

Step	What to check
1	Check for unclosed DB sessions -- confirm every get_db() dependency uses try/finally
2	Check for unbounded in-memory caches or lists that grow but are never evicted
3	Check background tasks that create objects (e.g. large lists) that outlive the request
4	Check for global variables/caches shared across requests that accumulate data
5	Use a memory profiler (tracemalloc, memory-profiler) in a staging environment under load

A very common root cause is exactly this pattern -- an in-process cache with no eviction policy, unintentionally used as a growing dictionary:

```

# BUG: this dict grows forever, one entry per unique request, never cleared
_cache = {}

@app.get("/lookup/{key}")
def lookup(key: str):

```

```
if key not in _cache:
    _cache[key] = expensive_computation(key)
return _cache[key]
```

The fix is to use a bounded cache with eviction (an LRU cache, or Redis with a TTL) instead of an unbounded process-local dictionary:

```
from cachetools import LRUCache

_cache = LRUCache(maxsize=1000)  # evicts oldest entries automatically

@app.get("/lookup/{key}")
def lookup(key: str):
    if key not in _cache:
        _cache[key] = expensive_computation(key)
    return _cache[key]

# Better for multi-instance deployments: externalize to Redis with a TTL
@app.get("/lookup/{key}")
async def lookup(key: str, redis: Redis = Depends(get_redis)):
    cached = await redis.get(key)
    if cached:
        return json.loads(cached)
    result = expensive_computation(key)
    await redis.set(key, json.dumps(result), ex=600)
    return result
```

Interview Answer:

Slow memory growth almost always traces back to something unbounded: an in-process dict used as a cache, unclosed DB sessions, or objects retained by background tasks. Diagnose with a profiler under sustained load, then replace unbounded in-memory structures with a bounded LRU cache or an external cache like Redis with a TTL, which also scales correctly across multiple instances.

Q15. Scenario: During a flash sale, your /checkout endpoint starts returning 500 errors and the database shows 'too many connections'. Walk through the root cause and the fix.

This is a well-known real-world incident pattern for e-commerce and ticketing platforms. Traffic spikes 20x during the sale, and the database's connection limit (often set around 100-200 by the hosting provider) gets exhausted within minutes.

The investigation should check, in order:

Check	Why it matters
Is pool_size + max_overflow across all app instances greater than the DB's max_connections?	If 10 instances each request a pool of 20, that's 200 possible connections against a DB capped at 150
Are sessions being closed properly on every code path, including exceptions?	Leaked sessions hold connections open even after the request finishes
Is the checkout route doing several sequential blocking DB calls that could be batched?	Each held-open connection during slow sequential calls reduces pool availability for other requests

Is there a queue/rate-limit in front of checkout to smooth the traffic spike?

Without backpressure, all 20x traffic hits the DB simultaneously

The immediate fix is usually a combination of a connection pooler (like PgBouncer) in front of Postgres -- so many app-level connections multiplex onto fewer real DB connections -- plus adding backpressure at the API layer so checkout requests queue instead of all hitting the DB at once:

```
# Add rate limiting specifically on the hot path during spikes
from slowapi import Limiter
limiter = Limiter(key_func=get_remote_address)

@app.post("/checkout")
@limiter.limit("5/second")    # smooths bursts instead of forwarding them all to the DB
async def checkout(order: OrderRequest, db: AsyncSession = Depends(get_db)):
    return await checkout_service.process(db, order)

# Use Redis to reserve inventory atomically before touching Postgres at all,
# so the database only sees requests that already passed a fast, in-memory check
async def reserve_stock(redis: Redis, product_id: str) -> bool:
    remaining = await redis.decr(f"stock:{product_id}")
    if remaining < 0:
        await redis.incr(f"stock:{product_id}")    # rollback
        return False
    return True
```

This reflects a broader production principle: protect the database as the scarcest resource. Absorb traffic spikes at the API/cache layer (rate limiting, Redis-based reservation, a message queue) rather than letting every request reach Postgres directly.

Interview Answer:

Connection exhaustion during traffic spikes is usually a combination of undersized pooling relative to instance count, leaked sessions, and no backpressure at the API layer. Fix it with a connection pooler like PgBouncer, ensure pool_size across all instances stays under the DB's connection limit, and add rate limiting or a fast in-memory reservation step (Redis) so the database only receives requests that have already passed initial validation.

Designing Good APIs -- SOLID Principles Explained (16-20)

Q16. (S) How does the Single Responsibility Principle guide good REST endpoint design, and what does a violation look like in practice?

A very common real-world violation: a single route handler validates input, runs business logic, sends an email, writes an audit log, and updates a cache -- all inline. It works, but it has many reasons to change: a change to email wording, audit format, or caching strategy all require editing the same function, and the route becomes untestable without mocking five unrelated systems.

```
# Violates SRP -- one function, five responsibilities
@app.post("/orders")
def create_order(order: OrderIn, db: Session = Depends(get_db)):
    new_order = Order(**order.dict())
    db.add(new_order)
    db.commit()
    send_email(order.customer_email, "Order confirmed")    # side effect 1
    write_audit_log(f"Order {new_order.id} created")      # side effect 2
    redis_client.delete(f"cache:orders:{order.customer_id}") # side effect 3
```



```
return new_order
```

The route's only responsibility should be translating an HTTP request into a service call and the result back into an HTTP response. Side effects that don't need to block the response belong in a background task or an event handler triggered after the core transaction commits:

```
@app.post("/orders", status_code=201)
def create_order(
    order: OrderIn,
    background_tasks: BackgroundTasks,
    db: Session = Depends(get_db),
    order_service: OrderService = Depends(get_order_service),
):
    new_order = order_service.create(db, order)          # one responsibility
    background_tasks.add_task(notify_order_created, new_order.id) # decoupled
    return new_order

def notify_order_created(order_id: int):
    send_email(...)
    write_audit_log(...)
    invalidate_cache(...)
```

This also makes the endpoint faster to respond (the client doesn't wait for email delivery) and easier to test -- you can assert `create_order()` calls `order_service.create()` without needing a real email or Redis connection.

Interview Answer:

A route handler's single responsibility is translating HTTP in and out. Business logic belongs in a service, and non-critical side effects (email, logging, cache invalidation) belong in background tasks or event handlers, not inline in the route. This keeps endpoints fast, focused, and easy to unit test.

Q17. (O) How does the Open/Closed Principle apply to evolving an API without breaking existing clients?

A common mistake when adding a new feature is modifying an existing endpoint's behavior in place -- e.g. changing what fields a response returns, or adding a required request field. Any client built against the old contract breaks immediately. The Open/Closed Principle says code (and by extension, an API contract) should be open for extension but closed for modification.

```
# BUG: adding a required field breaks every existing client immediately
class OrderIn(BaseModel):
    product_id: int
    quantity: int
    gift_wrap: bool          # newly added as required -- old clients now fail validation
```

The additive, backward-compatible fix is to make new fields optional with sensible defaults, and to introduce genuinely breaking changes behind a new API version rather than mutating the existing one:

```
# Correct: extend without breaking existing clients
class OrderIn(BaseModel):
    product_id: int
    quantity: int
    gift_wrap: bool = False  # optional, defaults preserve old behavior

# Breaking change -- ship as a new version instead of editing v1
```

```
app.include_router(orders_v1_router, prefix="/api/v1")
app.include_router(orders_v2_router, prefix="/api/v2")
```

The same principle applies to business logic that varies by condition -- e.g. discount rules per customer tier. Instead of an if/elif chain that must be edited for every new tier, a Strategy pattern lets you add a new tier as a new class:

```
class DiscountStrategy(ABC):
    @abstractmethod
    def apply(self, price: float) -> float: ...

class GoldDiscount(DiscountStrategy):
    def apply(self, price: float) -> float:
        return price * 0.9

class PlatinumDiscount(DiscountStrategy):
    def apply(self, price: float) -> float:
        return price * 0.8

# Adding a new tier later = one new class, zero edits to existing ones
```

Interview Answer:

Extend APIs additively -- new optional fields, new endpoints, new versions -- rather than mutating existing contracts that clients already depend on. For logic that varies by condition, prefer a strategy/plugin pattern over if/elif chains, so new behavior is added via new code, not by editing and re-testing code that already works.

Q18. (L) What does the Liskov Substitution Principle mean for Pydantic schemas and interchangeable service implementations?

LSP states that a subtype must be usable anywhere its base type is expected, without breaking the caller's assumptions. A subtle, common violation happens when a 'specialized' Pydantic schema adds stricter requirements than its parent, breaking code written against the parent:

```
class NotificationRequest(BaseModel):
    message: str
    recipient: str

class SmsNotificationRequest(NotificationRequest):
    recipient: str = Field(..., max_length=15) # phone numbers only

# Code written against the base type breaks for the subtype:
def send(req: NotificationRequest):
    notify(req.recipient, req.message) # assumes any string recipient is valid

send(SmsNotificationRequest(message="hi", recipient="not-a-phone-number-at-all"))
# passes base validation, then fails downstream in the SMS provider -- surprising
```

The same principle matters more practically for service implementations behind a shared interface. If one implementation secretly requires something the interface doesn't promise, code written against the interface breaks when that implementation is swapped in:

```
class NotificationSender(ABC):
    @abstractmethod
    def send(self, recipient: str, message: str) -> None: ...

class EmailSender(NotificationSender):
```

```

def send(self, recipient: str, message: str) -> None:
    smtp_client.send(recipient, message)    # works for any string

class SmsSender(NotificationSender):
    def send(self, recipient: str, message: str) -> None:
        if not recipient.startswith("+"):
            raise ValueError("SMS requires E.164 format") # stronger precondition!
        sms_client.send(recipient, message)

# Code that swaps senders via Depends() now breaks unexpectedly for SmsSender
# even though both satisfy the same abstract interface

```

The fix is to either enforce the stricter contract at the interface level (so every implementation genuinely honors it) or validate/normalize input before it reaches the interchangeable implementation, rather than letting one implementation silently demand more than the interface promises.

Interview Answer:

Any implementation substituted behind a shared interface -- a Pydantic subclass or a service implementing an abstract base class -- must not demand more or promise less than the base contract. If one implementation has stricter requirements, either encode that requirement in the shared interface itself, or validate for it explicitly before dispatching, so swapping implementations never causes a surprising failure.

Q19. (I) How does Interface Segregation shape request/response schema design and service interfaces so clients aren't forced to depend on things they don't need?

A common anti-pattern is a single, giant response model reused across every endpoint that touches a User -- profile pages, admin panels, and public listings all return the same UserFull schema, which forces every client to receive (and every consumer of the model to think about) fields it doesn't need, including sensitive ones.

```

# One bloated model used everywhere -- violates ISP
class UserFull(BaseModel):
    id: int
    name: str
    email: str
    hashed_password: str
    internal_notes: str
    last_login_ip: str
    is_admin: bool

@app.get("/users/{id}/public-profile", response_model=UserFull)    # way too much exposed
def public_profile(id: int): ...

```

The fix defines narrow, purpose-specific schemas per use case, so each endpoint only exposes what that specific client actually needs:

```

class UserPublicProfile(BaseModel):
    id: int
    name: str

class UserAdminView(BaseModel):
    id: int
    name: str
    email: str
    is_admin: bool

```

```

        last_login_ip: str

@app.get("/users/{id}/public-profile", response_model=UserPublicProfile)
def public_profile(id: int): ...

@app.get("/admin/users/{id}", response_model=UserAdminView)
def admin_view(id: int): ...

```

The same idea applies to service-layer interfaces: a single UserService with 20 methods (including admin-only operations like `ban_user` or `impersonate`) forces every consumer -- including simple read-only routes -- to depend on a large surface area. Splitting it into `UserReadService` and `UserAdminService` means a route that only lists users never even has access to destructive admin methods.

Interview Answer:

Design narrow, purpose-specific response models and service interfaces instead of one large shared model reused everywhere. Each endpoint or consumer should only see the fields and methods it actually needs -- this limits accidental data exposure and keeps the blast radius of any interface change small.

Q20. (D) Beyond Depends(), how does Dependency Inversion shape the overall architecture of an API from routers down to external systems?

Applied at the level of a single function, DIP means using `Depends()`. Applied at the architecture level, it means the entire dependency direction of the application points inward, toward abstractions -- this is often called Hexagonal or Ports-and-Adapters architecture. High-level business logic (the service layer) never imports a concrete database driver, HTTP client, or message queue library directly -- it depends only on an interface (a 'port'), and a concrete 'adapter' is plugged in at startup.

```

# Port (abstraction) -- lives in the domain/service layer, knows nothing about SQL
class OrderRepository(ABC):
    @abstractmethod
    def save(self, order: Order) -> None: ...
    @abstractmethod
    def get(self, order_id: int) -> Order | None: ...

# Adapter (concrete implementation) -- lives in the infrastructure layer
class PostgresOrderRepository(OrderRepository):
    def __init__(self, db: Session):
        self.db = db
    def save(self, order: Order) -> None:
        self.db.add(order)
        self.db.commit()
    def get(self, order_id: int) -> Order | None:
        return self.db.query(Order).get(order_id)

# Service depends only on the port, never the adapter
class OrderService:
    def __init__(self, repo: OrderRepository):
        self.repo = repo
    def place_order(self, order: Order) -> None:
        # pure business logic, zero SQL, zero FastAPI, zero HTTP knowledge
        self.repo.save(order)

# Wiring happens only at the edge, via Depends()
def get_order_service(db: Session = Depends(get_db)) -> OrderService:

```

```
return OrderService(PostgresOrderRepository(db))
```

The practical payoff: OrderService can be unit tested with an in-memory fake repository and zero database, it can be moved to a different database engine by writing one new adapter class, and it could even be reused outside of FastAPI entirely -- e.g. in a Celery worker or a CLI script -- because it has no framework dependency at all.

Interview Answer:

At the architecture level, Dependency Inversion means business logic depends only on abstract interfaces ('ports'), while concrete infrastructure (Postgres, Redis, external APIs) implements those interfaces as swappable 'adapters' wired in at the edges via Depends(). This keeps the core of the application framework-agnostic, database-agnostic, and fast to unit test.

System Design Fundamentals Every Backend Engineer Should Know (21-27)

Q21. What is the difference between horizontal and vertical scaling, and why does statelessness matter for scaling an API?

This is one of the most fundamental system design distinctions, and it directly determines how an API must be written.

	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
What it means	Add more CPU/RAM to one machine	Add more machine instances behind a load balancer
Ceiling	Hard physical/cost limit on one box	Effectively unlimited
Downtime for upgrade	Usually requires a restart/resize	None -- new instances join the pool live
Failure impact	One machine down = full outage	One instance down = load balancer routes around it

Horizontal scaling only works if any instance can handle any request -- which requires the API to be stateless. A common mistake is storing session data or an in-process cache in a local Python dict:

```
# BUG: breaks the moment you run more than one instance
active_sessions = {} # lives only in this process's memory

@app.post("/login")
def login(creds: Credentials):
    session_id = create_session()
    active_sessions[session_id] = creds.user_id # only visible to THIS instance
    return {"session_id": session_id}
```

If a load balancer routes the next request to a different instance, that instance has never heard of this session_id, and the user is incorrectly logged out. The fix is to externalize any shared state to a service every instance can reach:

```
@app.post("/login")
async def login(creds: Credentials, redis: Redis = Depends(get_redis)):
    session_id = create_session()
    await redis.set(f"session:{session_id}", creds.user_id, ex=3600) # shared by all instances
    return {"session_id": session_id}
```

Interview Answer:

Vertical scaling adds resources to a single machine and has a hard ceiling; horizontal scaling adds more machines and scales further, but requires the application to be stateless. Any session data, cache, or in-memory state must live in a shared store like Redis, not in a local Python variable, or horizontal scaling silently breaks.

Q22. What are the main caching strategies (cache-aside, write-through, write-behind), and when should each be used?

Caching is one of the highest-impact, lowest-effort performance techniques -- but picking the wrong strategy causes stale data bugs that are hard to track down. The three classic strategies:

Strategy	How it works	Best for
Cache-aside (lazy loading)	App checks cache first; on a miss, reads DB and populates cache	Read-heavy data that tolerates a short TTL, e.g. product listings
Write-through	App writes to cache and DB at the same time, synchronously	Data that must always be fresh in cache, e.g. inventory counts
Write-behind (write-back)	App writes to cache immediately; DB write happens asynchronously later	Very high write throughput where some risk of data loss is acceptable

```
# Cache-aside -- the most common pattern in typical CRUD APIs
@app.get("/products/{id}")
async def get_product(id: int, redis: Redis = Depends(get_redis), db=Depends(get_db)):
    cached = await redis.get(f"product:{id}")
    if cached:
        return json.loads(cached)

    product = await db.get(Product, id)
    await redis.set(f"product:{id}", product.json(), ex=300)  # 5 min TTL
    return product

@app.put("/products/{id}")
async def update_product(id: int, data: ProductUpdate, redis: Redis = Depends(get_redis), db=Depends(get_db)):
    product = await product_service.update(db, id, data)
    await redis.delete(f"product:{id}")  # invalidate rather than trying to update the cache
    return product
```

A key rule of thumb: on writes, it's usually safer to invalidate (delete) the cache entry rather than try to update it in place -- updating risks the cache and DB drifting out of sync if the update logic differs even slightly.

Interview Answer:

Cache-aside (read-through on miss, invalidate on write) covers most read-heavy API endpoints and is the safest default. Write-through suits data that must never be stale in cache. Write-behind maximizes write throughput at the cost of a small durability window. When in doubt, invalidate the cache key on write rather than trying to keep it in sync.

Q23. What is the CAP theorem, and how does it practically influence database choices in a backend system?

The CAP theorem states that a distributed data store can only guarantee two of three properties at the same time, during a network partition: Consistency (every read sees the latest write), Availability (every request gets a response), and Partition Tolerance (the system keeps working despite network failures between nodes).

Since network partitions are a fact of life in any distributed system, partition tolerance is effectively mandatory -- so the real-world choice in practice is between prioritizing Consistency or Availability when a partition occurs.

System type	Typical choice	Example use case in a backend
CP (Consistency + Partition tolerance)	Reject/delay requests rather than risk stale data	Payments, inventory counts, account balances -- PostgreSQL, etcd
AP (Availability + Partition tolerance)	Always respond, even if the data might be slightly stale	Activity feeds, view counts, product recommendations -- Cassandra, DynamoDB

In practice, most real backends are not single-database systems -- they deliberately mix consistency models per use case. A typical e-commerce backend uses a strongly consistent Postgres for the order and payment tables (correctness matters more than speed), while using an eventually-consistent store or a cache for less critical data like 'recently viewed products':

```
# Payments -- correctness matters more than raw speed; strongly consistent DB
async def charge_customer(db: AsyncSession, order_id: int, amount: float):
    async with db.begin():
        # strict transaction, no partial state
        await payment_repository.deduct_balance(db, order_id, amount)

# Recently viewed -- staleness is acceptable, availability matters more
async def track_view(redis: Redis, user_id: int, product_id: int):
    await redis.lpush(f"recent:{user_id}", product_id) # fire-and-forget, eventually consistent
```

Interview Answer:

CAP theorem says you can't have perfect consistency and availability during a network partition -- you must choose. Payment and inventory data typically needs a CP system (like Postgres) because incorrect data is worse than a slow response. Non-critical, high-volume data (feeds, view counts, recommendations) typically favors an AP system, because availability matters more than perfect freshness.

Q24. What is the difference between database read replicas and sharding, and when would you reach for each?

As traffic grows, a single database instance eventually becomes the bottleneck. There are two fundamentally different scaling techniques, and confusing them is a common mistake in system design interviews and real capacity planning:

	Read Replicas	Sharding
Solves	Too many reads for one instance to handle	Too much data / too many writes for one instance to hold
How	Copy the full dataset to N read-only replicas	Split the dataset across N instances by a shard key
Writes	Still all go to a single primary	Distributed across shards

Complexity added	Replication lag (eventual consistency on reads)	Cross-shard queries and joins become hard
------------------	---	---

A read replica setup is usually the first, simplest scaling step -- route writes to the primary and reads to replicas:

```
# SQLAlchemy: route reads and writes to different database URLs
write_engine = create_async_engine(PRIMARY_DB_URL)
read_engine = create_async_engine(REPLICA_DB_URL)

async def get_read_db():
    async with AsyncSession(read_engine) as session:
        yield session

async def get_write_db():
    async with AsyncSession(write_engine) as session:
        yield session

@app.get("/products")
async def list_products(db: AsyncSession = Depends(get_read_db)):
    ... # safe to serve from a replica, small staleness is fine

@app.post("/products")
async def create_product(data: ProductIn, db: AsyncSession = Depends(get_write_db)):
    ... # must go to the primary
```

Sharding is a bigger architectural commitment, usually reached for only once a single primary can no longer hold the data or handle the write volume even with good indexing -- e.g. partitioning users across databases by `user_id % N` or by geographic region.

Interview Answer:

Read replicas solve a read-throughput problem by copying data to read-only followers, at the cost of slight replication lag. Sharding solves a data-volume or write-throughput problem by partitioning data across multiple primaries, at the cost of much higher complexity (cross-shard queries, rebalancing). Reach for replicas first -- they're simpler -- and only shard when write volume or data size genuinely requires it.

Q25. What are message queues used for in a backend system, and what's the difference between point-to-point and pub-sub messaging?

A message queue decouples the producer of work from the consumer that does it, which solves three very common real-world problems: absorbing traffic spikes (buffer requests instead of dropping them), letting slow operations run without blocking the API response, and letting multiple services react to the same event independently.

Pattern	How it works	Example
Point-to-point (queue)	Each message is consumed by exactly one worker	Processing a video upload -- only one worker should transcode it
Publish-subscribe (pub/sub)	Each message is delivered to every subscriber independently	'Order placed' event -- inventory, email, and analytics services all react

```
# Point-to-point with Celery -- exactly one worker processes each task
@app.post("/videos/{id}/process")
def process_video(id: int):
    transcode_video.delay(id) # one worker picks this up
```



```

    return {"status": "queued"}

# Pub/sub with Redis -- every subscriber gets a copy of the event
@app.post("/orders")
async def create_order(order: OrderIn, redis: Redis = Depends(get_redis)):
    new_order = await order_service.create(order)
    await redis.publish("order.created", new_order.json())
    # inventory-service, email-service, and analytics-service
    # are all subscribed to "order.created" independently
    return new_order

```

In practice: RabbitMQ and Celery are common for point-to-point task queues (background jobs, report generation). Kafka is commonly used for pub/sub event streaming at high volume, because it retains a durable, ordered log that multiple independent consumer groups can replay.

Interview Answer:

Message queues decouple producers from consumers, letting an API respond quickly while heavier work happens asynchronously. Point-to-point queues (Celery/RabbitMQ) ensure exactly one worker handles each job -- ideal for background tasks. Pub/sub (Kafka, Redis pub/sub) broadcasts each event to every interested service independently -- ideal for triggering multiple, unrelated reactions to the same business event.

Q26. What are the main rate-limiting algorithms, and how do token bucket and sliding window differ in practice?

Rate limiting protects an API from abuse and traffic spikes, but different algorithms have very different behavior at the edges -- an interview follow-up almost always asks about the tradeoffs, not just 'use a library'.

Algorithm	Behavior	Downside
Fixed window	Counts requests in fixed time blocks (e.g. per minute)	Allows a burst of 2x the limit right at the window boundary
Sliding window	Counts requests in a rolling time window	More accurate, slightly more computation/memory
Token bucket	Bucket refills at a steady rate; each request consumes a token	Allows controlled bursts up to the bucket size -- often desirable
Leaky bucket	Requests processed at a strictly constant rate	Smooths bursts completely, but adds latency under load

Token bucket is the most commonly used in production APIs because it allows short, legitimate bursts (a user clicking around quickly) while still enforcing a steady long-term rate. A minimal Redis-backed implementation:

```

async def check_rate_limit(redis: Redis, key: str, capacity: int = 10, refill_rate: float = 1.0) -> bool:
    now = time.time()
    bucket = await redis.hgetall(key)
    tokens = float(bucket.get("tokens", capacity))
    last_refill = float(bucket.get("last_refill", now))

    tokens = min(capacity, tokens + (now - last_refill) * refill_rate)
    if tokens < 1:
        return False    # rate limited

```

```

    await redis.hset(key, mapping={"tokens": tokens - 1, "last_refill": now})
    await redis.expire(key, 60)
    return True

@app.get("/api/data")
async def get_data(request: Request, redis: Redis = Depends(get_redis)):
    allowed = await check_rate_limit(redis, f"rate:{request.client.host}")
    if not allowed:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    return {"data": "ok"}

```

Interview Answer:

Fixed window is simplest but allows boundary bursts up to 2x the limit. Sliding window fixes that at some extra cost. Token bucket is the most common production choice because it allows short bursts while enforcing a steady average rate. Leaky bucket enforces a perfectly constant output rate, trading burst tolerance for smoothness.

Q27. What is an API Gateway, and which responsibilities belong there versus inside individual backend services?

In a microservice architecture, a common design mistake is either putting zero cross-cutting logic in front of services (every service reimplements auth, rate limiting, and logging independently) or, at the other extreme, putting business logic inside the gateway itself, turning it into a hidden monolith no one can reason about.

An API Gateway is a single entry point that sits in front of backend services and handles cross-cutting concerns that are the same for every request, regardless of which service ultimately handles it:

Belongs in the Gateway	Belongs in the Service
TLS termination	Business logic and validation rules
Authentication (verifying who the caller is)	Authorization for specific resources (can THIS user edit THIS order)
Rate limiting and request throttling	Database access
Routing to the correct backend service	Domain-specific error handling
Request/response logging and metrics	Core application state

```

# Gateway layer -- generic, applies to every downstream service identically
@app.middleware("http")
async def gateway_middleware(request: Request, call_next):
    if not verify_jwt(request.headers.get("Authorization")):
        return JSONResponse(status_code=401, content={"detail": "Unauthorized"})
    if not await check_rate_limit(request.client.host):
        return JSONResponse(status_code=429, content={"detail": "Too many requests"})
    response = await call_next(request)
    log_request(request, response)
    return response

# Individual service -- only knows its own domain logic
@app.get("/orders/{id}")
async def get_order(id: int, user=Depends(get_current_user)):
    order = await order_service.get(id)

```

```

if order.customer_id != user.id and not user.is_admin:
    raise HTTPException(403) # resource-specific authorization, not the gateway's job
return order

```

The practical benefit: adding a new microservice automatically gets authentication, rate limiting, and logging for free by sitting behind the existing gateway, without reimplementing any of it.

Interview Answer:

An API Gateway centralizes cross-cutting concerns that are identical across every service -- authentication, TLS, rate limiting, routing, and logging -- so individual services don't reimplement them. Business logic, resource-specific authorization, and domain rules stay inside each service. Confusing the two either duplicates infrastructure code everywhere or turns the gateway into an unmanageable hidden monolith.

API Paradigms: REST vs GraphQL vs gRPC vs Real-Time (28-32)

Q28. What are the fundamental differences between REST, GraphQL, and gRPC, and how do you decide which one to use for a given service?

These aren't interchangeable style choices -- each optimizes for a different problem, and picking the wrong one causes real pain later (a public API built in gRPC is unusable from a browser without a proxy; a high-throughput internal service built in REST+JSON pays a real serialization tax).

	REST	GraphQL	gRPC
Data format	JSON over HTTP/1.1 or 2	JSON over HTTP, single endpoint	Protocol Buffers (binary) over HTTP/2
Client control over response shape	Fixed per endpoint	Client specifies exact fields needed	Fixed per RPC method (defined in .proto)
Best for	Public APIs, simple CRUD, wide client compatibility	Complex UIs with varied/nested data needs (mobile + web sharing one API)	Internal service-to-service calls needing low latency
Browser support	Native	Native (just HTTP POST)	Requires grpc-web proxy
Streaming	Limited (SSE/WebSocket needed separately)	Subscriptions (via WebSocket)	Native bidirectional streaming

A plain REST endpoint in FastAPI returns a fixed shape regardless of what the client actually needs:

```

@app.get("/users/{id}")
def get_user(id: int):
    return {"id": id, "name": "...", "email": "...", "address": {...}, "orders": [...]}
# Every client gets the full object, even a mobile screen that only needs "name"

```

A GraphQL resolver (using Strawberry, a popular FastAPI-friendly library) lets the client request only what it needs in a single round trip:

```

import strawberry
from strawberry.fastapi import GraphQLRouter

@strawberry.type
class User:
    id: int

```

```

    name: str
    email: str

@strawberry.type
class Query:
    @strawberry.field
    def user(self, id: int) -> User:
        return get_user_from_db(id)

schema = strawberry.Schema(query=Query)
app.include_router(GraphQLRouter(schema), prefix="/graphql")

# Client query -- requests ONLY name, nothing else is sent over the wire
# query { user(id: 1) { name } }

```

A gRPC service definition is language-agnostic and generates strongly-typed client/server code, making it a common choice for internal microservice-to-microservice calls where both ends are backend services you control:

```

// user.proto
service UserService {
    rpc GetUser (UserRequest) returns (UserResponse);
}
message UserRequest { int32 id = 1; }
message UserResponse { int32 id = 1; string name = 2; string email = 3; }

```

Interview Answer:

REST is the default for public, simple, widely-compatible APIs. GraphQL fits products where different clients (mobile, web, partners) need different slices of deeply nested data and over/under-fetching is a real problem. gRPC fits internal, high-throughput service-to-service communication where both ends are backend systems and binary serialization plus HTTP/2 streaming outweigh the cost of browser incompatibility.

Q29. What are over-fetching and under-fetching, and how does GraphQL solve them compared to REST?

Over-fetching happens when a REST response includes far more data than the client needs -- a mobile app showing just a username downloads the user's full address, order history, and preferences too, wasting bandwidth.

Under-fetching is the opposite and often worse: a single REST endpoint doesn't return enough related data, forcing the client to make several follow-up requests.

```

# Under-fetching example: rendering an order summary page requires 3 round trips
GET /orders/42          # returns order, but not customer name or product details
GET /customers/7        # now fetch the customer separately
GET /products/19        # now fetch each product separately
# 3 requests, 3 round trips of network latency, just to render one screen

```

A common REST workaround is creating bespoke aggregate endpoints for each screen (e.g. /order-summary/42), but this leads to endpoint sprawl -- a new one for every UI variation, each needing separate versioning and maintenance.

GraphQL solves both problems by letting the client describe exactly the shape of data it needs, resolved server-side in a single request, regardless of how deeply nested the relationships are:

```

# One request, one round trip, exactly the fields the screen needs
query {
  order(id: 42) {

```

```

    id
    total
    customer { name }
    products { name, price }
  }
}

```

The tradeoff: GraphQL pushes complexity from the client (making multiple calls) to the server (resolvers must be written carefully to avoid their own N+1 problem -- fetching each nested field with a separate DB query per item, which is solved with batching tools like DataLoader).

Interview Answer:

Over-fetching wastes bandwidth by returning unused fields; under-fetching forces multiple round trips to assemble one screen's data. GraphQL fixes both by letting clients request an exact, arbitrarily nested shape in one request -- at the cost of needing careful resolver design (batched loading) to avoid moving the N+1 problem from the client to the server.

Q30. How do you choose between WebSockets, Server-Sent Events (SSE), and polling for real-time features in a backend?

A common mistake is reaching for WebSockets for every 'live' feature, even when a much simpler mechanism would do -- WebSockets add real operational complexity (sticky sessions, connection state, reconnect logic) that isn't free.

	Polling	Server-Sent Events (SSE)	WebSockets
Direction	Client asks repeatedly	Server pushes to client (one-way)	Full duplex, both directions
Complexity	Very low	Low -- plain HTTP, auto-reconnect built in	Higher -- persistent connection, custom protocol
Good for	Infrequent updates, simplicity over freshness	Live feeds, notifications, progress updates (server -> client only)	Chat, multiplayer, collaborative editing (bidirectional)
Browser support	Universal	Universal (native EventSource API)	Universal, but needs a WS-aware load balancer

For a one-way live feed (e.g. order status updates, live scores), SSE is often the simplest correct choice -- plain HTTP, works through normal load balancers, and browsers auto-reconnect on drop:

```

@app.get("/orders/{id}/status-stream")
async def order_status_stream(id: int):
    async def event_generator():
        while True:
            status = await get_order_status(id)
            yield f"data: {json.dumps({'status': status})}\n\n"
            await asyncio.sleep(2)
    return StreamingResponse(event_generator(), media_type="text/event-stream")

```

Reach for a full WebSocket connection only when the client also needs to send data back over the same persistent connection (e.g. a chat message, a live cursor position in a collaborative doc) -- that's the case SSE and polling genuinely cannot handle.

Interview Answer:

Use polling for simple, infrequent updates where a little staleness is fine. Use Server-Sent Events for one-directional live updates (server pushes to client) -- it's simpler and more infrastructure-friendly than WebSockets. Reserve WebSockets for genuinely bidirectional real-time interaction like chat or collaborative editing, since they carry real added operational complexity.

Q31. Scenario: You're designing a system with a public-facing API, internal microservices, and a live dashboard. How do you decide which API style to use where?

This is a realistic, composite system design question -- real production systems rarely use just one API style everywhere. A sensible design applies the right paradigm per boundary, based on who's calling and what they need:

Boundary	Callers	Recommended style	Why
Public/partner-facing API	Third-party developers, mobile & web clients with different data needs	REST (or GraphQL if client data needs vary a lot)	Wide compatibility, easy to document, versioned contracts partners can rely on
Internal microservice-to-microservice	Your own backend services, e.g. Order Service calling Inventory Service	gRPC	Low latency, strongly-typed contracts, efficient binary serialization, native streaming
Live dashboard / real-time metrics	Browser client watching data update continuously	Server-Sent Events or WebSocket	Server-initiated push avoids constant polling overhead
Third-party webhooks (e.g. payment provider notifying you)	External systems calling into you asynchronously	REST endpoint receiving POST callbacks	Simple, universally supported, easy for any external system to implement

A common architectural pattern that ties this together is an API Gateway that exposes a single REST or GraphQL surface to the outside world, while translating those calls into gRPC calls to internal services -- external consumers never need to know gRPC exists:

```
# Public-facing gateway endpoint -- simple REST for external clients
@app.get("/api/v1/orders/{id}")
async def get_order(id: int, inventory_client: InventoryGrpcClient = Depends(get_inventory_client)):
    order = await order_service.get(id)
    # internally, fan out to other services via fast, typed gRPC calls
    stock = await inventory_client.check_stock(order.product_id)
    return {"order": order, "in_stock": stock.available}
```

Interview Answer:

Different boundaries in the same system often justify different API styles: REST or GraphQL at the public edge for compatibility and flexibility, gRPC between your own internal services for speed and strong typing, and SSE/WebSockets for anything that needs server-initiated real-time updates. An API Gateway is the common pattern that lets you mix these without exposing internal protocol choices to external clients.

These questions reflect patterns seen in real production FastAPI codebases -- not just framework trivia -- and are commonly discussed in mid-to-senior backend engineering interviews.